Prerequisites and objectives

- Prerequisites are binary heaps, linked trees, induction, logarithms, and amortized analysis.
- Derive structures from meld requirements. Trace leftist, binomial, Fibonacci, and standard two-pass pairing heaps.
- Distinguish actual work, worst-case bounds, and amortized bounds. State whether decrease-key already has a valid node handle.

Mergeable priority queue contract

- A heap represents a multiset, allowing duplicate keys. Meld combines all elements of two heaps while preserving element identities.
- We use destructive meld. Source heaps no longer own transferred nodes afterward. Keeping independently mutable source copies has different costs.
- Decrease-key uses a valid node handle and lowers that node's key. Searching for an arbitrary key cannot be hidden inside an O(1) or O(log n) bound.
- With tied minima, any minimum may be returned. Stable order requires a composite key such as key plus insertion sequence number.

Leftist heaps: make the path used by meld short

Concatenating two binary-heap arrays and rebuilding takes linear time. A leftist heap uses a linked binary tree: **the whole tree need not be shallow; following right pointers must reach null quickly.** Meld descends through one right child at each level, so that is the path we constrain.

Set null rank to zero. For a nonempty node,

$$r(x) = 1 + \min\{r(x.left), r(x.right)\}.$$

A leaf has rank 1. A node with only a left child also has rank 1. Rank measures the shortest path to a null child position, **not height or child count**.

Maintain min-heap order and $r(left) \geq r(right)$. Thus $r(x) = 1 + r(x.right)$: rank counts nonempty nodes on the right spine. An n-node left-only chain is valid, with rank 1 and height n−1.

Meld: keep the smaller root, repair on return

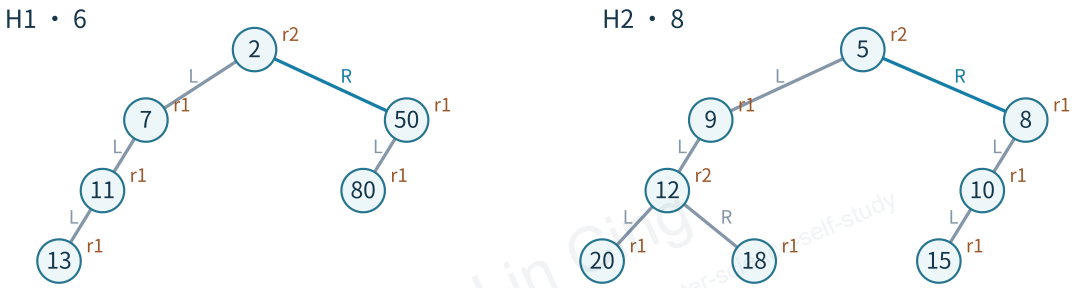
Inputs must be valid heaps with disjoint node sets. This destructive meld keeps the first root on a tie. Nodes store key, left, right and rank, with no parent pointers.

```
rank(null) = 0
meld(a, b):
    if a == null: return b
    if b == null: return a
    if a.key > b.key: swap(a, b)
    a.right = meld(a.right, b)
    if rank(a.left) < rank(a.right):
        swap(a.left, a.right)
    a.rank = 1 + rank(a.right)
    return a
```

Compare keys on descent; compare ranks on return. Keep a's left subtree and recursively meld its right subtree with the other heap. The new right rank is known only after that call returns. Swap pointers, never keys or payloads.

Meld inputs: 14 nodes and two right spines

Circles contain keys; r labels give ranks. L and R identify child pointers. An omitted edge is null, with rank 0. Blue edges mark the original right spines; other edges are not traversed by this recursive descent.



H1 has 6 nodes and spine 2→50. H2 has 8 nodes and spine 5→8. Both roots have rank 2. Node 9 has a deep left subtree but rank 1, because its right pointer is null.

Recursive descent: do not swap children yet

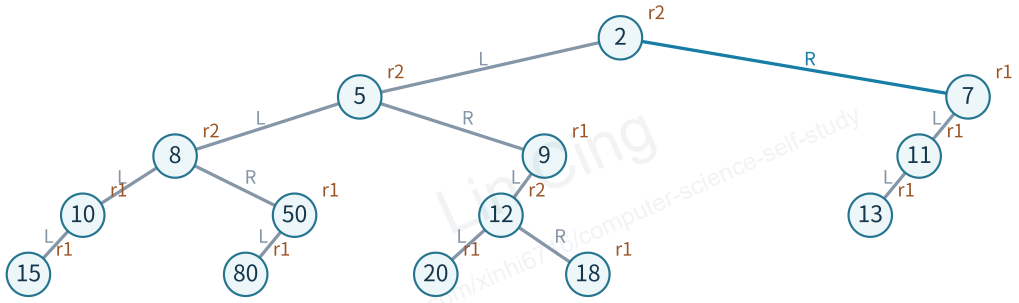
Keep the smaller root and recurse through its right child. Retained-root ranks have not yet received their return-time updates.

Call	Comparison and retained root	Left subtree kept	Next call
meld(2,5)	2<5: keep 2	Root 7	meld(50,5)
meld(50,5)	5<50: exchange local roots, keep 5	Root 9	meld(8,50)
meld(8,50)	8<50: keep 8	Root 10	meld(null,50)
meld(null,50)	Empty base case; no key comparison	—	Return root 50

There are **three root-key comparisons**. Calls at 2,5,8 remain pending. A temporarily assembled min-heap is not yet a repaired leftist heap. Unwind in order 8→5→2.

Unwinding: why roots 5 and 2 swap children

Returning root	Attached left/right roots	Left/right ranks	Action and new rank
8	10 / 50	1 / 1	No swap; r(8)=2
5	9 / 8	1 / 2	Swap: left 8, right 9; r(5)=2
2	7 / 5	1 / 2	Swap: left 5, right 7; r(2)=2



Why meld is correct

Induct on the total input size. If one heap is empty, returning the other preserves the node set and both invariants. Otherwise select the smaller root a. Recursion processes only a's right subtree and the other heap, strictly reducing total size.

1. **No lost or duplicated nodes.** Disjoint inputs partition into a, its original left subtree and the recursive inputs. Induction preserves recursive nodes; attachment and child swaps create or delete none.
2. **Heap order.** Root a is no greater than any key in its own subtrees or the other heap. It is no greater than the new child roots. The child heaps are valid by induction; exchanging their sides preserves these comparisons.
3. **Leftist property.** Both children are valid heaps. Put the higher-rank child on the left. Assigning $r(a) = 1 + r(a.right)$ now agrees with the defining minimum.

Strictly decreasing size also proves termination. meld(H,H) violates disjointness and may create a cycle. The ownership interface must exclude it; this operation does not copy the heap.

Worst-case cost: follow the original right spines

First bound rank. A subtree of rank at least r has two child subtrees of rank at least r−1. If N(r) is the minimum size for rank at least r, N(0)=0 and induction gives

$$N(r) \geq 1 + 2N(r-1) \geq 1 + 2(2^{r-1} - 1) = 2^r - 1.$$

Hence an n-node heap has $r \leq \lfloor \log_2(n+1) \rfloor$. The base case r=0 holds.

During descent no return-time swaps have occurred. Each step replaces one root by its **original right child**, reducing the two current ranks' sum by exactly one. When both heaps are nonempty that sum is at least two. Input ranks r1,r2 therefore permit at most r1+r2−1 nonempty levels, plus one empty base case.

Constant work per level gives **worst-case** time and stack space $O(1 + \log(n_1 + 1) + \log(n_2 + 1))$. The 14-node example has rank sum four and exactly three comparisons; other inputs can finish earlier.

Delete-min: meld the two child heaps

Remove root 2 from the 14-node result and meld roots 5 and 7. Reject deletion on an empty heap. Removing a singleton returns null.

Call or return	Decision
meld(5,7)	Keep 5 and left subtree 8; recurse on meld(9,7)
meld(9,7)	Keep 7 and left subtree 11; recurse on meld(null,9)
meld(null,9)	Return 9 with all its descendants
Return to 7	Children 11,9 both have rank 1; no swap; r(7)=2
Return to 5	Children 8,7 both have rank 2; no swap; r(5)=3

Result: `5:(8,7)`, `8:(10,50)`, `7:(11,9)`; all other edges remain. The right spine is 5→7→9, minimum is 5, and size is 13. **Rank can increase after deletion**: it measures shape and is not monotone in element count.

Insertion and rank conventions

Insert creates a singleton with `left=right=null, rank=1` and melds it with the heap. Insert key 1 into the preceding 13-node heap: keep 1, and meld(null,5) returns the old heap. The new root has left rank 0 and right rank 3. Swap children, placing the old heap on the left. The new root has rank 1. This uses two calls and one comparison, without searching through the old nodes.

Insert and delete-min take worst-case $O(1 + \log(n + 1))$; find-min reads a nonempty root in O(1). Allocation and arbitrary-precision key costs are excluded.

Here **null rank=0 and leaf rank=1**. Under null npl=−1 and leaf npl=0, the same position has $npl = r - 1$. Its right spine has npl+1 nonempty nodes; its subtree has at least $2^{npl+1} - 1$ nodes. Convert formulas and base cases together.

This differs from binomial/Fibonacci degree (direct child count) or $\log_2 size$ in splay analysis. These are different quantities, not universally convertible by shifting one. The right-spine bound does not bound overall height.

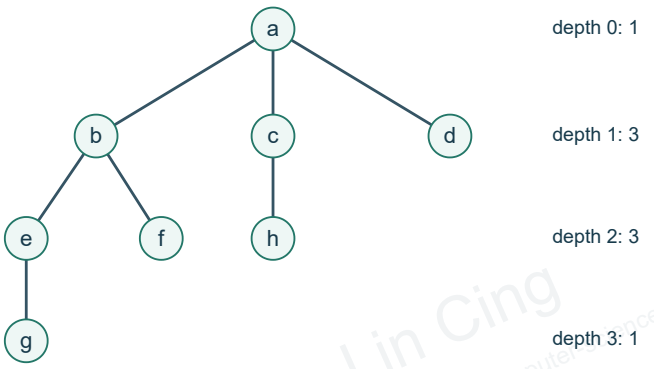
Binomial trees B_k

- B_0 is a singleton. Form B_k from two B_{k-1} trees by making one root a child of the other.
- Induction gives 2^k nodes, height k, root degree k, and child subtrees B_0 through B_{k-1}.
- A binomial heap stores heap-ordered binomial trees with at most one root of each degree.
- The binary expansion of n determines the degrees. For $n=13=1101_2$, the forest contains B_0, B_2, B_3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why binomial-tree level counts are binomial coefficients

Nodes at each depth of B_3



Link two B_2 trees: each new level count is old count plus the preceding count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Induction for binomial-tree levels

Let $b(k, d)$ count nodes at depth d in B_k, with the root at depth 0 and out-of-range counts equal to zero. The base tree B_0 has one root, so $b(0, 0) = 1$.

Link two B_{k-1} trees. Depths in the winner's old tree stay the same; every node in the other tree moves one level deeper. Hence

$$b(k, d) = b(k-1, d) + b(k-1, d-1) = \binom{k}{d}.$$

The final equality follows from Pascal's identity and the induction hypothesis. Summing levels gives $\sum_{d=0}^{k} \binom{k}{d} = 2^k$. For B_3, counts 1,3,3,1 total 8. This exact-shape statement does not directly apply to Fibonacci trees whose nodes may lose children through cuts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial meld as binary carrying

- Merge root lists by degree. For two equal-degree trees, compare roots and link the larger root below the smaller.
- Two B_k trees become B_{k+1}, like a binary carry. With three equal-degree trees, retain one and link the other two.
- At most $\lfloor \log_2 n \rfloor + 1$ roots exist. Standard eager consolidation meld takes worst-case O(log n).
- Melding sizes $5 = 101_2$ and $3 = 011_2$ carries through degrees 0,1,2, leaving one B_3 with eight nodes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial linking and dedicated insertion

```
binomial_link(a,b):
    require degree(a)==degree(b)
    if a.key>b.key: swap(a,b)
    prepend b to a.child list; b.parent=a
    a.degree+=1; return a
binomial_insert(x):
    carry=singleton(x); d=0
    while tree[d] exists: carry=link(carry,tree[d]); clear tree[d]; d+=1
    tree[d]=carry; update min pointer
```

`swap(a,b)` exchanges local root pointers, not node keys or payloads. Both inputs are Bk trees. The winner has child degrees k−1,…,0; prepending the losing Bk gives k,k−1,…,0, exactly the children of B(k+1). Keeping the smaller root preserves heap order on the new edge.

Binomial meld: input forests and list order

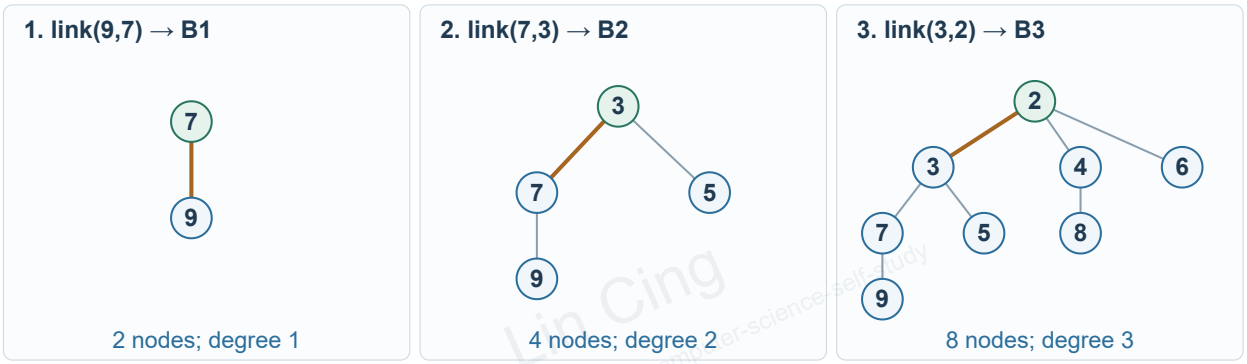
All keys are distinct. Notation `x:[child roots]` gives child-list order; unlisted nodes are leaves. H1 is `9:[], 2:[4,6], 4:[8]`; H2 is `7:[], 3:[5]`. Root lists use increasing degree; child lists use decreasing degree. A link makes the losing root the winner's **first child**.



H1 has 1+4=5 nodes; H2 has 1+2=3. The first root need not be the minimum: H1's minimum is 2, not 9.

Binomial meld: three comparisons and subtree links

Link B0 roots 9 and 7, then link with B1 root 3, then with B2 root 2. Each step compares only two roots and moves the entire losing subtree.



Brown marks the new parent→child edge; all other edges remain. Root 2 ends with children 3,4,6 of degrees 2,1,0. Each of the eight keys occurs once; no payload is exchanged or losing subtree discarded.

Binomial meld: what remains at each degree?

Step	Link	Remaining roots: key(degree)	Roots
Input	None	H1: 9(0),2(2); H2: 7(0),3(1)	4
degree 0	9 and 7 → 7	7(1),3(1),2(2)	3
degree 1	7 and 3 → 3	3(2),2(2)	2
degree 2	3 and 2 → 2	2(3)	1

One link removes one tree, not a node. Four roots become one in exactly three links. Linking equal B_k trees preserves the binomial shape and gives $2^k + 2^k = 2^{k+1}$ nodes. Keeping the smaller root preserves heap order on new and retained edges. The final degree 3 also matches $5{+}3{=}8{=}1000_2$.

Three links in this example do not establish O(1) general meld. The algorithm also visits input root lists, whose size is O(log n) for valid binomial heaps. A Fibonacci heap may have Θ(n) unconsolidated roots, so this root-list bound does not transfer.

Why not link three equal-degree trees into one?

For $7=111_2$ plus $3=011_2$, each degree receives at most two input roots and one carry.

degree	H1 roots	H2 roots	Incoming carry	Retained here	Outgoing carry
0	1	1	0	0	1
1	1	1	1	1	1
2	1	0	1	0	1
3	0	0	1	1	0

Three B1 trees have six nodes: retain one B1 and link the other two into B2. Six is not a power of two, so all three cannot form one binomial tree. The final B1 and B3 contain 2+8=10 nodes, matching 1010_2. Choosing a different retained tree can change the final shape; degree, key conservation, and heap order must still hold.

Conditions for other binomial operations

- A dedicated insertion adds B_0 and carries from low degrees, stopping as soon as carrying ends. Its worst case is O(log n), but carry accounting gives amortized O(1). Calling a meld that always scans the entire root list does not automatically obtain this insertion bound.
- Delete-min finds and removes the minimum root, reverses its child list into increasing degree order, then melds. Worst-case time is O(log n).
- Find-min costs O(log n) if it scans roots. A maintained minimum-root pointer makes it O(1).
- Decrease-key can exchange contents up the parent chain in O(log n). Node-identity handles require compatible handling of those content exchanges.

Binomial meld: carry through a degree table

```
meld_roots(R1,R2):
    by_degree=empty
    for x in concatenation of detached root snapshots R1,R2:
        while by_degree[x.degree] contains y:
            clear by_degree[x.degree]
            x=binomial_link(x,y)
        by_degree[x.degree]=x
    return roots in degree order; recompute minimum pointer
```

Each snapshot root enters once. On collision, clear the old degree slot, link, and retry at the new degree. Each link removes one root, so there are at most one fewer links than input roots. Valid input forests have O(log(n1+n2)) roots, giving O(log(n1+n2)) total visits and links. This root bound does not apply to unconsolidated Fibonacci heaps.

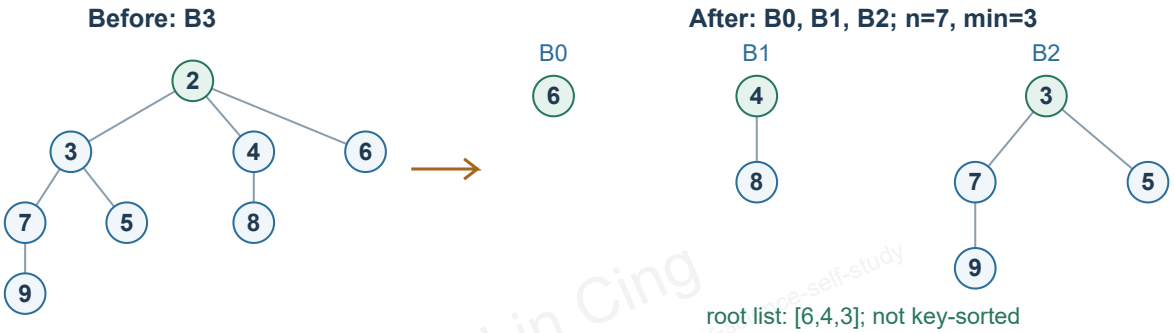
Binomial removal and decrease: restore parent and min

```
delete_min():
    require roots not empty
    z=minimum root
    remove z from roots
    children=detach z.children; clear their parent pointers
    release z
    roots=meld_roots(roots,children)
decrease_key(x,new):
    require new<=x.key; x.key=new
    while x.parent!=null and x.key<x.parent.key:
        swap complete key-value contents and update handle mapping
        x=x.parent
    update minimum pointer
```

Detach children and clear their parents before releasing the old root; meld recomputes min. Decrease-key may swap complete key-value contents, but logical-element handles require the handle-to-node mapping to follow those contents.

Delete-min: turn the same B3 back into a forest

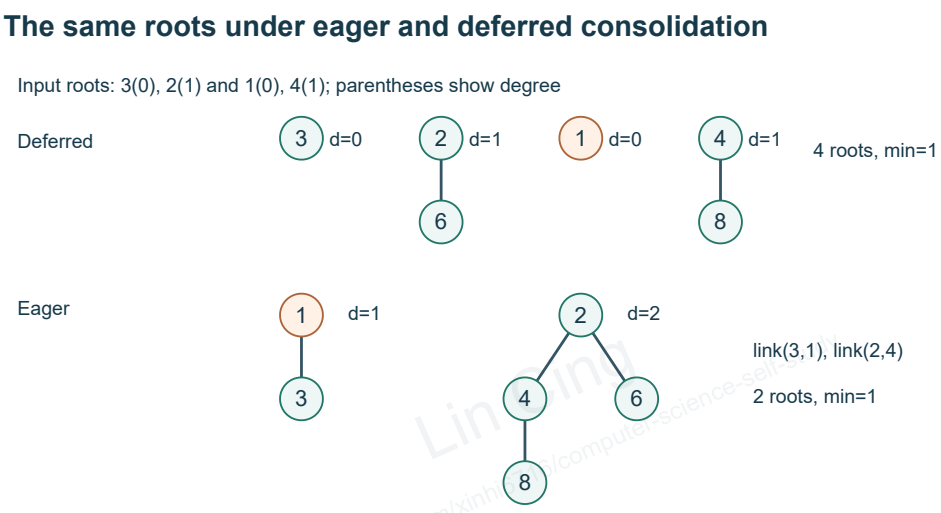
Remove 2 and keep its three subtrees. Reverse child degrees 2→1→0 into root degrees 0→1→2; clear each new root's parent.



Root list [6,4,3] holds 1+2+4=7 nodes and min=3. No other roots remain, so no link is needed.

Reverse by degree, not key. A degree-table variant can accept unreversed input.

Eager versus deferred consolidation



A deferred binomial-forest contract

Use exact binomial trees but permit multiple roots of the same degree. Maintain a doubly linked root ring and a minimum pointer. Meld concatenates disjoint root rings and compares minima; the source heaps relinquish ownership. Insert adds a singleton and updates the minimum without linking.

Delete-min removes the minimum, promotes its children, repeatedly links roots of equal degree, then rebuilds the root ring and minimum. Since every tree remains an intact B_k, each root degree is at most $\lfloor \log_2 n \rfloor$; after consolidation there is at most one root of each degree. This version does not promise the Fibonacci heap's decrease-key, mark, or cascade operations.

For the six-node roots 3(0),2(1),1(0),4(1), deferred meld keeps all four roots with min=1. One eager result links 3 with 1 and 2 with 4, retaining 1(1),2(2). Both contain the same six nodes, but only the eager version enforces unique degrees immediately after meld.

Root-count potential for the deferred version

Let t be the number of roots and $\Phi = A \cdot t$ for a fixed constant A large enough to pay per link and root scan. The empty heap has $\Phi = 0$. Insert adds one root, so its actual and amortized costs are O(1). Meld concatenates the root rings: the new root count is the sum of the old counts, so total potential is unchanged and actual and amortized costs are O(1). Both claims assume a maintained root ring and minimum pointer.

Before delete-min there are t roots and the removed root has degree d. After consolidation each degree has at most one root, so $t' \le \lfloor \log_2(n+1) \rfloor + 1$ and $d \le \lfloor \log_2 n \rfloor$. Actual work is at most $c(t+d+1)$, covering root scans, child promotion, links and rebuilding. The potential change is $A(t'-t)$. Choose $A \ge c$: the linear t term cancels, giving $O(1+\log(n+1))$ amortized cost. A single actual operation can still take $\Theta(n)$, because $\Theta(n)$ roots may have accumulated.

Without a cached minimum pointer, find-min must scan t roots and is not O(1). These bounds apply to this explicitly specified deferred variant, not an eager invariant requiring unique degrees after every meld.

Fibonacci heaps defer consolidation

- A Fibonacci heap has a heap-ordered forest, circular doubly linked root list, minimum-root pointer, and degree and mark fields per node.
- Insert adds a singleton root. Meld splices root lists and updates the minimum pointer. Neither immediately consolidates equal degrees.
- Delete-min promotes the removed root's children, links equal-degree roots using a degree table, then rebuilds the minimum pointer.
- Nodes can lose children, so trees do not retain fixed binomial shapes. Cascading cuts are needed for the degree bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decrease-key and cascading cuts

1. Lower x.key. If it remains at least its parent's key, no cut is needed.
2. Otherwise cut x from parent p, add x to the root list, and clear x.mark.
3. If p is not a root, mark it on its first lost child. If already marked, cut p and continue upward.

A nonroot node can lose at most one child after its latest attachment and still remain under that parent. Roots need not be marked on losing children.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci consolidation and mark operations

```
link_equal_degree_roots(a,b):
    require a.degree==b.degree
    if a.key>b.key: swap(a,b)
    roots=ring_remove(roots,b)
    a.child=ring_prepend(a.child,b)
    b.parent=a; b.mark=false; a.degree+=1
    return a
cut(x,p):
    p.child=ring_remove(p.child,x); p.degree-=1
    x.parent=null; x.mark=false; roots=ring_prepend(roots,x)
cascade(p):
    g=p.parent
```


Fibonacci consolidation and mark operations (continued)

```
    if g==null: return
    if not p.mark: p.mark=true
    else: cut(p,g); cascade(g)
decrease_key(x,new):
    require new<=x.key; x.key=new
    p=x.parent
    if p!=null and x.key<p.key: cut(x,p); cascade(p)
    if x.key<min.key: min=x
delete_min():
    require min!=null
    return extract_root(min)
```

Consolidating singleton roots [7,3,8,5] in that order puts 7 at degree 0. Root 3 links 7 below it to form degree 1. Root 8 occupies degree 0; root 5 links 8 below it, then collides with degree 1 root 3. Root 3 wins, with child 7 and child 5→8, degree 2. Iterate a snapshot or use mutation-safe root traversal while linking.

Fibonacci consolidation and mark operations (continued)

For marked nonroots p and g in a valid state, illustrate r=2→g=3→p=5→x=7 with g and p marked. Decreasing x to 1 cuts x, p, g into roots. Root count changes 1→4 and marked count 2→0, so Φ changes 5→4, ΔΦ=−1. Scaling potential pays fixed cut costs with constant endpoints left over. This state illustrates accounting only; the general degree bound still requires the attachment-time and one-lost-child proof.

Insert adds an unmarked singleton root and updates min. Destructive meld splices root cycles and transfers ownership. Empty delete-min rejects, and removal of the last node sets min=null. Arbitrary-node deletion uses the complete erase(handle) and extract_root procedures below without changing keys. A forced cut notifies the old parent through cascade even when heap order was not violated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete circular-list operations and shared consolidation

```
ring_remove(head,x):
    if x.right==x: head=null
    else:
        x.left.right=x.right; x.right.left=x.left
        if head==x: head=x.right
    x.left=x.right=x; return head
ring_prepend(head,x):
    require x.left==x and x.right==x
    if head!=null:
        x.left=head.left; x.right=head
        head.left.right=x; head.left=x
    return x
snapshot_of_root_ring():
    S=[]
    if roots==null: return S
    x=roots
    repeat: S.append(x); x=x.right
    until x==roots
```

Complete circular-list operations and shared consolidation (continued)

```
    return S
consolidate():
    A=[]                          // missing degree entries mean null
    for x in snapshot_of_root_ring():
        if x.parent!=null: continue
        while len(A)<=x.degree: A.append(null)
        while A[x.degree]!=null:
            y=A[x.degree]; A[x.degree]=null
            x=link_equal_degree_roots(x,y)
            while len(A)<=x.degree: A.append(null)
        A[x.degree]=x
    roots=null; min=null
    for x in nonnull entries of A:
        x.left=x.right=x; x.parent=null; x.mark=false
        roots=ring_prepend(roots,x)
        if min==null or x.key<min.key: min=x
```


Complete circular-list operations and shared consolidation (continued)

Ring_remove requires x to belong to the designated ring. A singleton yields null. Otherwise reconnect both neighbors and move the representative if necessary. Make the removed x a singleton before passing it to ring_prepend. Both operations change constantly many pointers without a predecessor search. Using them for root and child rings makes cut, promotion, and link explicit.

Snapshot root pointers before linking modifies the ring. Equal-degree roots compare keys; remove the larger root from roots, attach it to the smaller root's child ring, clear its mark, and increase the parent's degree, using link_equal_degree_roots from the earlier page. Clear the old table entry before carrying to the new degree. Each surviving table entry represents one valid heap-ordered tree.

Rebuild roots and min from the table without traversing the discarded old ring. Never read old min.key after releasing its node, or skip comparisons merely because z was not the old minimum. Grow the degree table as needed, using O(1+log(n+1)) entries. With t′ roots after cutting and target degree d, this snapshot version additionally stores O(t′+d) root pointers, worst-case O(n); auxiliary space is not just the degree table. One operation may also take Θ(n) actual time. Amortized time and instantaneous space are different claims.

Why Fibonacci degrees stay logarithmic

- Order x's k retained children by attachment time. When child i was linked, x already had at least i−1 earlier retained children. Equal-degree linking means the new child then had degree at least i−1.
- A child retained under x can have lost at most one child. Its current degree is at least max(0, i−2).
- A minimum-subtree-size recurrence then gives at least F_{k+2} nodes for degree k, hence degree O(log n).
- The bound connects permitted child losses to degree. Omitting cascading cuts invalidates this argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving the Fibonacci size bound

- Let S(k) be the minimum possible subtree size among nodes of degree at least k, including the root. This definition is nondecreasing in k. S(0)=1 and S(1)≥2.
- Any node of degree at least k has k children with degree lower bounds max(0, i−2). Thus for k≥2, $S(k) \geq 1 + 2S(0) + \sum_{j=1}^{k-2} S(j)$.
- Inductively substitute $S(j) \geq F_{j+2}$ to obtain $S(k) \geq 3 + \sum_{j=1}^{k-2} F_{j+2} = F_{k+2}$. The last identity is a Fibonacci prefix sum, with F_0=0, F_1=1.
- Since $F_{k+2} \geq \varphi^k$ for $\varphi = (1+\sqrt{5})/2 > 1$, every degree in an n-node heap satisfies $k \leq \log_\varphi n$. This justifies an O(log n) degree table.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential accounts for deferred work

- Use $\Phi = t + 2m$, where t counts roots and m counts marked nonroots. Amortized cost is $\hat{c} = c + \Phi_{\text{after}} - \Phi_{\text{before}}$.
- Insert increases roots by one and potential by a constant. Meld combines additive potentials and takes constant amortized work.
- Cutting a marked ancestor adds one root but clears one mark, reducing potential by one. Repeated cuts release potential to pay their actual work, apart from constant endpoint changes.
- Multiplying Φ by a sufficient fixed implementation constant pays for the constant pointer work per cut or link. Starting empty, nonnegative potential makes total actual work at most total amortized work plus initial potential. This is a worst-sequence guarantee, not a random average.

One Fibonacci potential and complete amortized bounds

For Fibonacci decrease-key with k≥1 cuts, root count rises by k. Beyond the first cut, each cut ancestor was marked; at most the final stopping ancestor acquires a new mark. Thus $\Delta m \le 1 - (k-1) = 2 - k$ and $\Delta(t + 2m) \le 4 - k$. If actual work is at most ak+b, choose scaled potential $A(t + 2m),\ A \ge a$. Amortized cost is at most $ak + b + A(4 - k) \le b + 4A$, independent of k. With no cut, key/minimum updates and comparisons are constant work.

For delete-min, let the initial root count be t, removed degree d, and maximum degree D. Actual work is at most a(t+d+D)+b. Promotion, linking, and clearing root marks do not increase the marked count. Consolidation leaves at most D+1 roots, so $\Delta\Phi \le A(D + 1 - t)$. Choosing A≥a yields amortized cost at most $a(d + D) + A(D + 1) + b = O(D + 1) = O(1 + \log(n + 1))$, since d≤D. This is the algebra paying for root scans; one actual operation may still cost Θ(n).

- Fibonacci heaps yield Dijkstra's abstract O(E+V log V) bound with valid handles and decrease-key, under Dijkstra's nonnegative-weight condition.

Erase without changing a key and extraction of an arbitrary root

```
erase(h):
    z=resolve_live_owned_handle(h)   // reject stale/foreign handle first
    if z.parent!=null:
        p=z.parent; cut(z,p); cascade(p)
    return extract_root(z)
extract_root(z):
    require z is a live root owned by this heap
    result=copy(z.item)             // fixed-size nonthrowing model
    while z.child!=null:
        c=z.child
        z.child=ring_remove(z.child,c); z.degree-=1
        c.parent=null; c.mark=false; roots=ring_prepend(roots,c)
    roots=ring_remove(roots,z); min=null
    invalidate_handle(z); release(z); n-=1
    if n==0: roots=null; return result
    consolidate()
    return result
```


Erase without changing a key and extraction of an arbitrary root (continued)

Each node stores key, item, parent, child, degree, mark, and circular-list left/right pointers. Roots and child designate root and child rings; null denotes an empty ring. A handle identifies a particular live node, not an arbitrary node with an equal key. Only the erased handle becomes invalid; other items and node identities are never swapped. Resolve an opaque external handle through its owning container's live-node registry before accessing a node. Reading an already released raw pointer to inspect an alive flag is invalid. Pointer-work bounds assume a valid handle; additional lookup/validation costs are separate.

For nonroot z, cut it into the root ring and notify its old parent p through cascade. An unmarked nonroot p becomes marked; a marked p is cut and the process continues upward; a root stops the cascade. This cut is required for deletion even without a key decrease. Omitting cascade would allow a nonroot to lose two children and invalidate the degree proof. A target already at a root enters extract_root directly.

Erase without changing a key and extraction of an arbitrary root (continued)

Extract_root does not require z=min. Remove each child c from z's ring before decrementing degree, clearing parent/mark, and adding its entire subtree to roots. Remove z, clear min to avoid a dangling pointer, invalidate its handle, release it, and decrement n exactly once. If nodes remain, consolidate all roots and recompute the actual minimum. No key is changed. Reject empty delete-min before access; an empty heap has no valid erase handle. Removing the last node gives n=0 and roots=min=null. Reusing that handle must fail. After destructive meld, accepted handles follow the container's ownership contract; the emptied source must not erase destination-owned nodes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

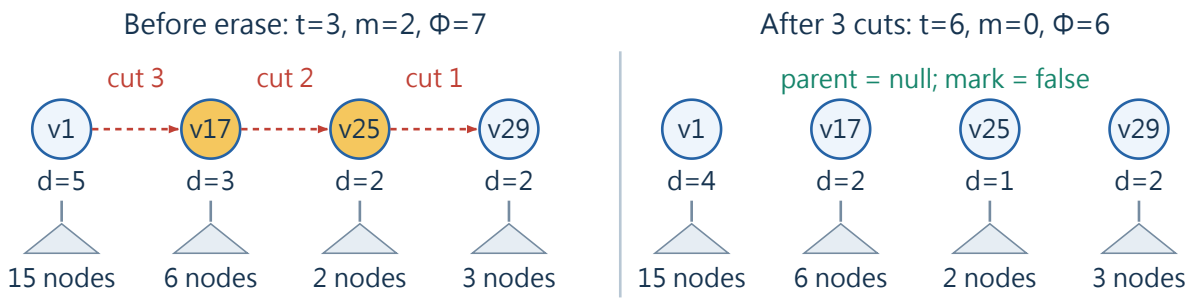
Legal erase setup and three cuts

```
for i=0..32: h[i]=insert(i)     // v_i is permanent identity, initially key i
delete_min()                    // removes v_0; consolidates to root v_1
decrease_key(h[18],-10)
decrease_key(h[26],-11)
erase(h[29])                    // key stays 29 until release
```

Prepend to root and child rings and traverse snapshots through right links. After inserting 0..32 and removing v0, root v1 has degree 5. Its child v17 originally has children v25,v21,v19,v18; v25 has v29,v27,v26; v29 has v31,v30; v31 has v32. Lowering v18 to −10 cuts it and marks v17. Lowering v26 to −11 cuts it and marks v25. Before erase, roots are v26(key−11,d0),v18(key−10,d0),v1(key1,d5), with marked nonroots v17,v25. Target v29 is not the minimum and still has two child subtrees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

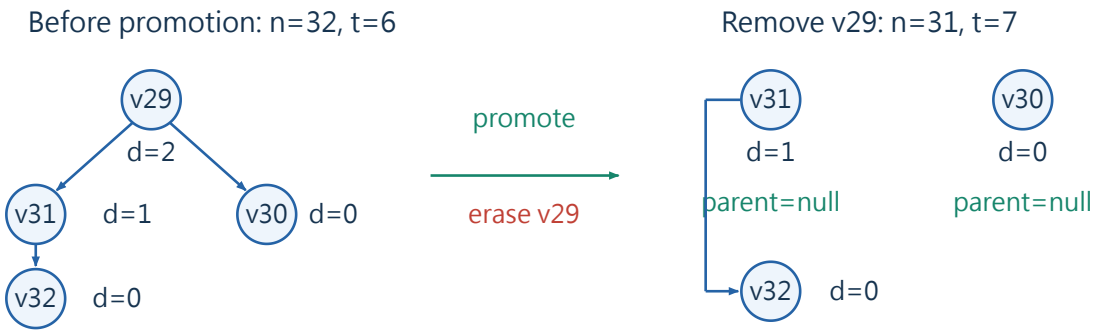
Legal erase setup and three cuts (continued)



Other roots: v18(key −10), v26(key −11). Gold = mark true.
Triangles retain child forests; counts exclude the parent. Both states have n=32.

The four displayed nodes, unchanged child forests of sizes 15,6,2,3, and two other roots account for all 32 nodes. Cutting changes parent links, not node identities. Each already marked ancestor becomes an unmarked root; root v1 is not marked when the cascade stops.

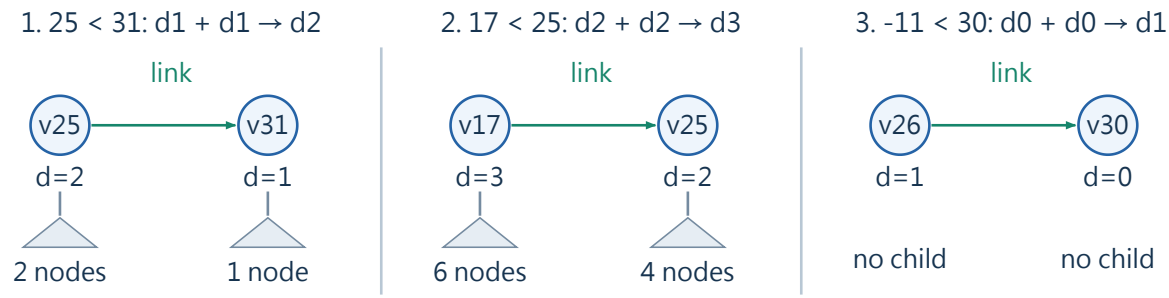
Promote target children and remove exactly one node



Unchanged roots: v1, v17, v25, v26, v18 (28 nodes); only h29 expires.

Moving v31 preserves its child v32, so it remains a two-node tree. Together with singleton v30, the promoted trees contain three nodes. Only v29 is removed; v26 with key −11 remains the minimum, and all surviving handles still identify their original nodes.

Three degree-table carry links



A[0]=v18, A[1]=v26, A[3]=v17, A[4]=v1; min=v26 (key −11)
Subtree sizes: 1 + 2 + 12 + 16 = 31; t=4, m=0, Φ=4.

The first link makes v25 a five-node tree of degree 2; the second attaches it to v17, producing degree 3 and twelve nodes. Linking v30 under v26 produces degree 1 and two nodes. Unchanged roots v18 and v1 have one and sixteen nodes, respectively.

Erase state table and identity conservation

Completed step	Roots t	Marked nonroots m	$\Phi = t + 2m$	Required change
Before erase	3	2	7	v29.parent=v25; v25.parent=v17
Cut v29	4	2	8	v25.degree falls 2→1
Cascading cut v25	5	1	7	Clear v25.mark; v17.degree falls 3→2
Cascading cut v17	6	0	6	Clear v17.mark; v1.degree falls 5→4
Promote v31,v30	8	0	8	Both parents become null; v29.degree=0
Remove/release v29	7	0	7	n falls 32→31; h29 is invalid
After three links	4	0	4	Distinct degrees; recompute min=v26

Erase state table and identity conservation (continued)

The root snapshot is v30(d0),v31(d1),v17(d2),v25(d1),v26(d0),v18(d0),v1(d4). Equal-degree v25 wins v31 and reaches degree 2; v17 then wins v25 and reaches degree 3. Root v26(key−11) wins v30 and reaches degree 1. Final table entries are 0→v18,1→v26,3→v17,4→v1. There are 31 nodes and the minimum remains key−11. Promoted v31,v30 later become children again, without changing node identities. Promotion describes an intermediate root state, not a promise to remain a root after consolidation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness and amortized derivation for arbitrary erase

```
// Potential Phi_A=A*(t+2*m), with a constant A large enough for pointer work.
// Cut/cascade phase: k cuts, actual <= a*k+b, Delta(t+2m)<=4-k.
// Root extraction: degree d, initial t' roots, final at most D(n)+1 roots.
// Actual <= a*(t'+d+D(n)+1)+b; marks never increase in this phase.
```

Correctness has three parts. Cutting z removes one parent edge and preserves every subtree's internal heap order. Cascading enforces the first-loss mark and second-loss cut rule needed by the degree proof. Promoting z's children preserves all their elements; only z.item is removed and only z's handle becomes invalid. Finally, linking equal-degree roots under the smaller root preserves order and node sets. Taking the least final root yields the global minimum, whether or not z was previously minimal.

If cutting uses k≥1 cuts, roots increase by k. Apart from the initial target, each further cut ancestor was marked, and at most one final ancestor acquires a new mark. Thus $\Delta m \le 2-k$ and $\Delta(t+2m) \le 4-k$. For actual cost≤ak+b, choose scaled potential $\Phi_A = A(t+2m)$, A≥a. Amortized cut cost is at most $ak+b+A(4-k) \le b+4A = O(1)$. A target already at a root needs only constant checks in this phase.

Correctness and amortized derivation for arbitrary erase (continued)

After cutting, let there be t′ roots and let z have degree d. Maximum degree during consolidation has bound $D(n) = O(\log(n+1))$. Extraction costs at most $a(t' + d + D(n) + 1) + b$. Promotion and linking never increase marks, and at most D(n)+1 roots survive, so $\Delta\Phi_A \leq A(D(n) + 1 - t')$. With A≥a and d≤D(n), amortized extraction is at most $a(d + D(n) + 1) + A(D(n) + 1) + b = O(1 + \log(n+1))$. The two potential differences telescope using the same potential, proving the complete erase bound.

One operation can still cost Θ(n), due to long cascading cuts or a large root scan/snapshot. Start-from-empty sequence analysis uses the same nonnegative Φ_A; arbitrary startup adds its initial value. Key/result operations and node release follow the fixed-cost model, and handle lookup is separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing heap link

- A standard pairing heap is a multi-child min-heap. Meld compares two roots and makes the larger a new child of the smaller, changing a constant number of pointers.
- Insert melds a singleton. Decrease-key usually cuts the lowered node with its subtree and melds it with the root.
- Constant-time removal from the sibling list requires sibling, parent, predecessor, or equivalent backlink information. A next-only list without a way to locate its predecessor may require scanning. Require a live handle owned by this heap before access.
- A small immediate pointer count does not justify assigning every operation O(1) amortized cost across a sequence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing sibling backlinks and detachment

```
// Pairing roots passed to link have parent=prev=next=null.
detach(x):
    p=x.parent
    if x.prev==null: p.first_child=x.next
    else: x.prev.next=x.next
    if x.next!=null: x.next.prev=x.prev
    x.parent=x.prev=x.next=null
link(a,b):
    if a==null: return b
    if b==null: return a
    if a.key>b.key: swap(a,b)
    b.parent=a; b.prev=null; b.next=a.first_child
    if b.next!=null: b.next.prev=b
    a.first_child=b; return a
```

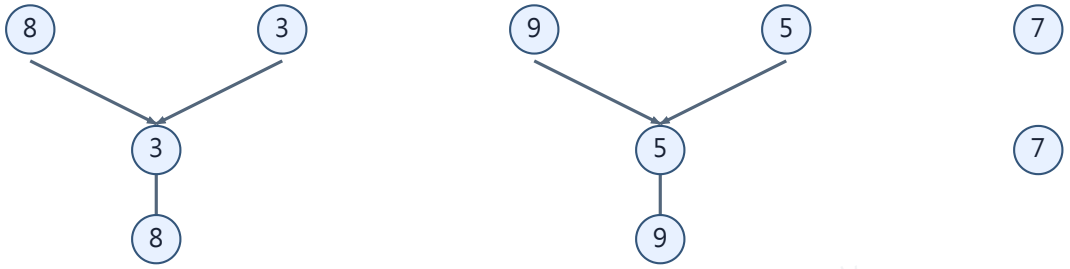
Each node stores parent, previous/next siblings, and first_child. Detach applies only to a nonroot. The first child's prev is null; other prev links point to the previous sibling. Removing the first child updates its parent's first_child, while removing another updates the previous sibling's next. Both cases repair the next sibling's prev. The entire subtree below x remains intact. Lowering the root needs no detach. Before delete-min pairs children, save each next pointer and clear parent/prev/next so every child becomes a detached root.

Complete two-pass pairing and deferred costs

```
link(a,b):
    if a==null: return b
    if b==null: return a
    if a.key>b.key: swap(a,b)
    prepend b to a.children; return a
delete_min():
    require root!=null
    children=detach_all_children(root); release(root)
    paired=[]
    for adjacent (a,b) left-to-right: paired.append(link(a,b))
    if odd last child remains: paired.append(last)
    root=null
    for heap in reverse(paired): root=link(heap,root)
decrease_key(x,new):
    require new<=x.key; x.key=new
    if x!=root: detach x in O(1) using backlinks; root=link(root,x)
```

Complete two-pass pairing and deferred costs (continued)

Pairing first pass: adjacent pairs; unpaired 7 remains



Roots 8,3,9,5,7 pair (8,3) into 3 with child 8, (9,5) into 5 with child 9, leaving 7. Right-to-left accumulation links 7 with empty, then 5 with 7 to obtain children 7,9, then 3 with 5 to obtain children 5,8. Subtree 5 retains 7,9. No paired subtree is dismantled. Immediate delete-min link work is proportional to the old root degree and can be Θ(n).

Actual subtree shapes during the pairing second pass

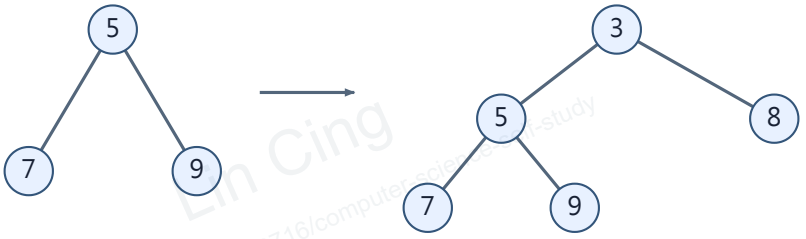
```
paired=[root3(child8),root5(child9),root7]
acc=link(root7,null); acc=link(root5,acc)
acc=link(root3,acc)       // 4 nonempty links across both passes
```

Second pass folds right to left, preserving entire subtrees

link(5,7) → 5



link(3,5) → 3

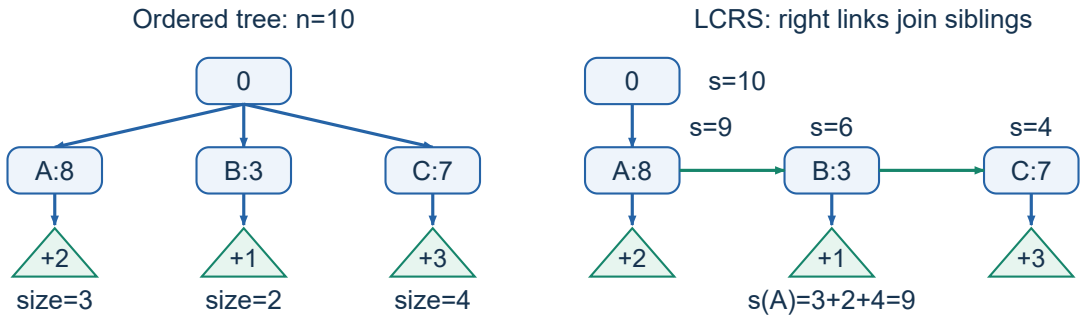


LCRS representation and a nonnegative potential

View the ordered multiway tree as a left-child/right-sibling (LCRS) binary tree. Let s(v) count all nodes reachable from v through these two binary links, $\rho(v) = \log_2 s(v)$, and $\Phi = \sum_v \rho(v)$. This includes later sibling subtrees: **s(v) is not v's original multiway subtree size**. Every s(v)≥1, so Φ≥0; an empty heap has Φ=0. These are analysis variables, not metadata recomputed during operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LCRS representation and a nonnegative potential (continued)



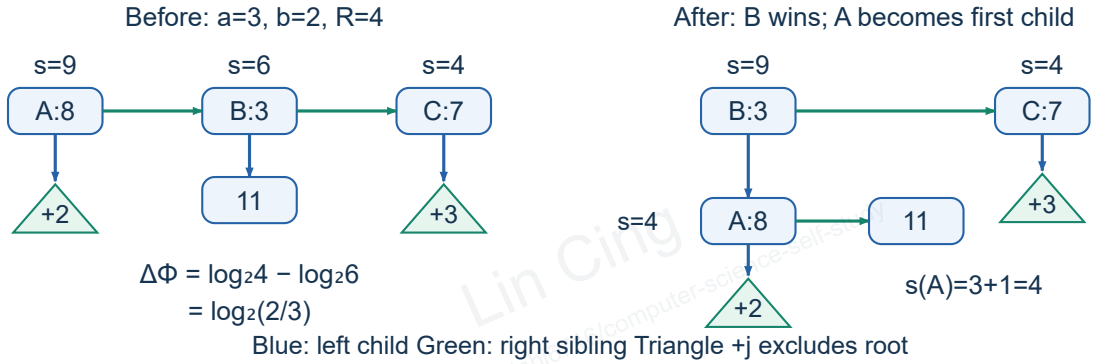
Triangle +j: j collapsed descendants; the displayed root is excluded

Root 0 has child subtrees of sizes 3,2,4. In the binary representation s(A)=9,s(B)=6,s(C)=4, and s(root)=10. Removing the root leaves the ordered forest of children: only the root's $\log_2 10$ disappears; every other s is unchanged.

For independent live heaps, total potential is the sum of their separate potentials; roots are not sibling-linked. During delete-min, define a temporary forest with virtual right-sibling adjacency in component order. If an array implementation clears actual next pointers while detaching roots, interpret the virtual chain from this component order. Do not assign unrelated heap potentials to each cleared-pointer intermediate state. Each macro-link includes constant detach/reconnect work; scanning, temporary storage, and releasing the old root cost O(k+1). Both endpoint potentials match the actual heaps.

An adjacent link changes only two root ranks

Consider adjacent components A,B in one forest, with original multiway sizes a,b≥1 and a suffix containing R≥0 nodes. Before linking, s(A)=a+b+R and s(B)=b+R. Link makes the losing root the winner's **first child**. Resolve equal keys consistently.



After linking, the winner has s=a+b+R and the loser has s=a+b−1. All other sizes are unchanged: a root in the prefix reaches the same suffix nodes, and old child chains retain their contents. Either winner therefore gives

$$\Delta\Phi_{\text{link}} = \log_2(a+b-1) - \log_2(b+R)$$

First-pass inequality with a positive suffix

Write A=a+b and T=A+R. If R>0, the exact adjacent-link formula gives

$$1 + \Delta\Phi_{\text{link}} = \log_2 \frac{2(A-1)}{b+R} \le \log_2 \frac{2A}{R} \le 2\log_2 \frac{T}{R}$$

The first inequality uses A−1≤A and b+R≥R. The second is equivalent to $2AR \le (A+R)^2$, whose right side minus left side is $A^2 + R^2 \ge 0$. The argument depends only on sizes, not which root key wins.

Number the pairs processed left-to-right. Let $T_j = a_j + b_j + R_j$ be the size still present from pair j to the right end. The untouched suffix of one pair is precisely the total starting at the next pair, so $R_j = T_{j+1}$. Summing the logarithms cancels intermediate terms. A final pair with R=0 cannot use this division by R and is handled separately below.

Telescoping the first pass for odd degree

Before deletion there are n nodes; after removing the root N=n−1 remain. Suppose its degree is k=2q+1. The first pass makes q links and leaves the rightmost tree of size r≥1 unpaired. This tree lies in every processed pair's suffix, so every $R_j > 0$.

$$q + \Delta\Phi_1 \le 2\sum_{j=1}^{q} \log_2 \frac{T_j}{R_j} = 2\log_2 \frac{N}{r} \le 2\log_2 N$$

Consequently $\Delta\Phi_1 \le -q + 2\log_2 N$. The negative −q term is needed to pay for a linear number of links. A bound saying only that potential increases by O(log n) would omit this payment.

In the existing five-singleton example 8,3,9,5,7, N=5,q=2,r=1. The two exact link changes are $\log_2(1/4)$ and $\log_2(1/2)$, totaling −3. For k=1, q=0 and r=N: the empty sum is 0, and no link occurs. No log0 is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The final zero-suffix pair for even degree

If k=2q≥2, the last pair has $R_q = 0$. Write $A_q = a_q + b_q \ge 2$. The first q−1 pairs telescope to

$$(q-1) + \Delta\Phi_{\text{previous}} \le 2\log_2(N/A_q)$$

Use the exact formula for the last pair: $\Delta\Phi_{\text{last}} = \log_2(A_q - 1) - \log_2 b_q \le \log_2 A_q$. Therefore

$$\Delta\Phi_1 \le -q + 1 + 2\log_2(N/A_q) + \log_2 A_q \le -q + 1 + 2\log_2 N$$

When q=1 the earlier sum is empty and $N = A_q$, so its upper bound is 0. A uniform odd/even bound is $\Delta\Phi_1 \le -q + 2\log_2 N + 1$, where q=⌊k/2⌋. Degree zero is a separate singleton case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Second pass and the complete delete-min bound

The first pass leaves m=⌈k/2⌉ trees. Accumulate right-to-left. If the next left tree has size a and the accumulated right tree has size b, then R=0, and

$$\Delta\Phi_{\text{link}} = \log_2(a+b-1) - \log_2 b \le \log_2 \frac{a+b}{b}$$

The next accumulator size is a+b, so $\Delta\Phi_2 \le \log_2(N/b_0) \le \log_2 N$, where $b_0 \ge 1$ is the rightmost component size. If m≤1 there are no links. The five-child example has second-pass ratios 2 and 4/3; including the first pass and removal of the original root gives total potential ratio (1/6)(1/4)(1/2)(2)(4/3)=1/18.

For k≥2, combine root removal and both passes: $\Delta\Phi \le -\log_2 n - q + 3\log_2 N + 1 \le -q + 3\log_2 n + 1$. Actual work is at most ck+d for fixed constants, and k≤2q+1. Choose the same scaled potential $\Psi = C\Phi$ with $C \ge 2c$. $\text{actual} + \Delta\Psi \le (2c-C)q + 3C\log_2 n + c + d + C = O(1+\log n)$. This pays for scanning and all k−1 links, not just first-pass links.

For k=0, n=1 and deletion leaves an empty heap: ΔΦ=0 and actual cost O(1). For k=1, the sole child becomes root, $\Delta\Phi = -\log_2 n$, and actual cost is O(1). Reject empty delete-min. The array procedure stores O(k) roots, possibly O(n) space; the proof does not imply logarithmic temporary space.

Meld, insert, and decrease under the same potential

Meld two independent nonempty heaps of sizes a,b, with n=a+b. Initially their root ranks are $\log_2 a$ and $\log_2 b$. Afterward the winner has s=n and the loser s=n−1; other sizes do not change. Thus

$\Delta\Phi_{\text{meld}} = \log_2 \frac{n(n-1)}{ab} \le \log_2 n$, since $ab \ge a+b-1 = n-1$. An empty operand gives ΔΦ=0. Inserting into a nonempty heap melds a singleton and has exactly $\Delta\Phi = \log_2 n$; inserting the first node has ΔΦ=0. Although both operations take O(1) immediate work, **this analysis charges O(1+log(n+1)) amortized for each**.

Decreasing the root's key does not change shape or potential. For a nonroot x of original multiway subtree size a, first cut it into a separate heap. The binary size of x loses its right-sibling suffix. Binary ancestors that previously reached the removed subtree only lose reachable nodes; other sizes are unchanged. Thus cutting has ΔΦ≤0. Melding it back adds at most $\log_2 n$. A valid handle and the stated parent/prev/next backlinks make the actual cut O(1). The same Ψ therefore charges decrease-key O(1+log(n+1)); it does not establish constant amortized decrease-key.

Sequence cost and the scope of this analysis

Operation	Immediate actual cost	Amortized charge under this same Ψ
find-min	O(1)	O(1)
insert	O(1)	O(1+log(n+1))
meld	O(1)	O(1+log(n+1)), n combined size
decrease-key	O(1), valid handle/backlinks required	O(1+log(n+1))
delete-min	O(k+1), worst-case O(n)	O(1+log(n+1))

Starting with empty heaps, $\Psi_0=0$ and the final $\Psi \geq 0$. Hence $\sum c_i = \sum \hat{c}_i + \Psi_0 - \Psi_{\text{final}} \leq \sum \hat{c}_i$. If N is the maximum live-node count and I,M,D,K,F count insertions, melds, deletions, decreases, and find-min operations, total cost is $O((I + M + D + K)(1 + \log_2(N + 1)) + F)$, plus O(1) for each empty-heap creation. A nonempty initial state contributes Ψ_0.

This proves one compatible collection of bounds for the standard two-pass variant. Substituting a different analysis's constant amortized insert/meld into this proof does not establish $O(I + M + (D + K) \log N + F)$. Constant immediate cut work also does not imply constant amortized decrease-key. Changing pass directions or the prepend-child rule requires checking the inequalities again.

Complexity comparison and selection

Structure	find-min	meld	delete-min	decrease-key
Leftist	O(1) worst	O(log n) worst	O(log n) worst	No fast bound supplied here
Eager binomial	O(1) with minimum pointer	O(log n) worst	O(log n) worst	O(log n) worst
Fibonacci	O(1) worst	O(1) worst	O(log n) amortized	O(1) amortized
Two-pass pairing	O(1) worst	O(1) immediate pointer work	O(log n) amortized	O(log n) amortized under this potential

Every decrease-key bound assumes a valid handle. Amortized bounds for different operations must belong to one compatible analysis. Independently selecting optimistic bounds can produce an invalid combination.

Keep constant terms for empty and singleton cases. Melding two empty heaps still tests pointers and takes O(1); the precise leftist bound is O(1+log(n1+1)+log(n2+1)). Delete-min requires nonempty input. Reject an empty request instead of invoking log 0.

Rank, degree and size in context

- Leftist rank counts nodes on the short right spine under the null-rank-zero convention; it is not the height of the whole tree.
- Binomial and Fibonacci degree counts direct children. A binomial B_k has exactly 2^k nodes; a Fibonacci tree can lose children, so it needs a separate size lower bound.
- The pairing-heap analysis uses $\log_2$ of an LCRS-reachable size as a potential. It is an analysis quantity, not a degree field or a leftist rank.

Keep each definition with its own invariant before comparing the cost bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Operation contracts to check

Situation	Required behavior
find-min or delete-min on an empty heap	Report empty; do not dereference a null root
Equal keys	Any minimum is valid; each handle still identifies the correct item
Destructive meld	Sources relinquish ownership; input heaps must not share nodes
decrease-key / erase	Use a live handle; deletion invalidates that handle
Amortized bound	Use one compatible potential and account for initial potential

Distinguish actual work of one operation from amortized cost over a sequence.

References and attribution

Lin Cing, handwritten data structures notes; Fibonacci Heaps; Pairing Heaps; Sinnamon–Tarjan.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study